

# Project ALMA — Project & Architecture Report

AI-Powered Alumni Management & Career Intelligence Platform

**System:** ALMAconnect Full-Stack Ecosystem

**Target:** Students • Alumni • College Administrators

**Tech Stack:** React 19 • Node.js • Express • Python ML • MongoDB

## Project ALMA — Comprehensive Project & Architecture Report

AI-Powered Alumni Management & Career Intelligence Platform

### 1. Project Overview

#### 1.1 Project Name & Brand

- **Repository / Project Name:** Project ALMA ( [darshanemd/ALMAconnect](#) / [project-alma](#) )
- **Portal Identity:** ALMAconnect — Institutional Alumni Networking & Career Intelligence Portal

#### 1.2 Purpose & Problem It Solves

Higher education institutions consistently struggle with three persistent, systemic challenges:

1. **Disconnected Alumni Networks:** Graduated students quickly lose touch with their alma mater and fellow peers. Institutional directories are outdated, static spreadsheets or forgotten email lists, resulting in near-zero active engagement and lost mentorship opportunities.
2. **Career Preparedness & Placement Uncertainty:** Students lack empirical, data-driven insight into whether their academic performance (CGPA, backlogs), technical competencies (DSA, projects, internships), and resumes align with corporate hiring standards across Tier 1 (Product), Tier 2 (Growth Tech), and Tier 3 (Enterprise) organizations.
3. **Administrative Overhead & Fragmented Systems:** College administrators spend countless manual hours verifying graduating batches, handling paper-based complaints, distributing circulars, onboarding alumni, and organizing campus reunions without automated verification, unified ticketing, or digital verification cards.

**Project ALMA** solves these problems by providing an integrated, full-stack, AI-augmented ecosystem. It connects **Students, Alumni,** and **College Administrators** in a single multi-tenant platform. By combining real-time communication (Socket.IO), machine learning engines (XGBoost placement prediction and TF-IDF NLP resume intelligence), digital verification cards with QR code security, anonymous grievance redressal, and automated lifecycle services (e.g. auto-graduation), ALMA transforms institutional relationship management from a reactive chore into an intelligent career accelerator.

### 1.3 Target Users

| Target User Group      | Role Key      | Primary Responsibilities & Interaction                                                                                                                                                                                                                         |
|------------------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| College Students       | student       | Evaluates placement readiness via ML simulators, analyzes resumes against ATS benchmarks, explores Alumni Digital Twins, seeks 1:1 alumni mentorship, applies for jobs, submits anonymous complaints, downloads digital student ID cards, and RSVPs to events. |
| Graduated Alumni       | alumni        | Networks in global directory, provides 1:1 mentorship and resume reviews, posts jobs and referral openings, shares knowledge through technical blogs, creates teams for hackathons, and accesses digital alumni privilege smart cards.                         |
| College Administrators | college_admin | Verifies and approves student/alumni registrations with automated credential emails, performs bulk CSV imports, broadcasts targeted campus circulars, builds dynamic surveys, resolves institutional grievances, and monitors campus-wide analytics.           |

### 1.4 Main Objectives

- **Data-Driven Placement Intelligence:** Provide instantaneous, transparent placement probability scores and hiring tier predictions using ensemble ML algorithms trained on real engineering placement datasets.
- **ATS Resume Scoring & Automated Gap Analysis:** Extract text from PDF/DOCX resumes, compute ATS compatibility scores, pinpoint missing domain keywords, and offer a live, print-ready resume builder.
- **Frictionless Administrative Operations:** Automate alumni registration approval workflows with multi-channel email dispatch (Gmail SMTP, Brevo, Resend, SendGrid), pre-verified student registries, and bulk CSV ingestion.
- **Real-Time Collaboration:** Power 1-on-1 private messaging, active presence indicators, typing feedback, and push notifications with zero polling delay.
- **Secure Community Environment:** Provide encrypted authentication, RBAC, campus isolation, contact masking, and an anonymous grievance redressal system with zero submitter identity leakage.

## 2. Complete Feature List

Below is the exhaustive catalog of every feature present across the application, categorized by user perspective and where it is accessed:

### 2.1 Public & Authentication Features

1. **Public Landing Page & Real-Time Pulse ( / ):**
  - Displays live institutional statistics fetched directly from MongoDB (verified alumni count, student count, active job openings, upcoming events).
  - Features a real-time activity ticker showcasing recently posted jobs, upcoming events, and newly connected alumni.
  - Interactive hero showcase highlighting platform benefits and dynamic login/registration CTAs.
2. **Multi-Role Registration Portal ( /register ):**
  - Context-aware registration form for students, alumni, and college administrators.
  - Dynamic department, graduation year, degree, roll number, and ID proof document upload.
  - Cross-references incoming student registrations against the PreVerifiedStudent institutional registry to provide instant verification if pre-approved.

### 3. Secure Authentication & Session Management ( </login> ):

- JWT-based authentication supporting both HTTP-only cookies and Authorization Bearer headers.
- Role-based automatic dashboard dispatching.
- Comprehensive "Forgot Password" workflow utilizing time-limited 6-digit OTP codes or secure cryptographic tokens sent via SMTP.

## 2.2 Student Career & AI Intelligence Tools

### 4. Ensemble ML Placement Predictor & Stress Simulator ( </student/placement-predictor> ):

- Multi-feature input panel: CGPA, Backlogs, Internships, Technical Aptitude, DSA depth, Academic Tier, and Target Job Role.
- Runs an ensemble model (XGBoost + Random Forest soft voting) via a persistent Python daemon worker to output placement probability (%) and hiring tier (Product Tier 1, Tech Tier 2, Enterprise Tier 3).
- SHAP-style feature importance radar charts and interactive "What-If" stress testing sliders.

### 5. AI ATS Resume Analyzer & Parser ( </student/resume-analyzer> ):

- Client-side parser for [.pdf](#) (via [pdfjs-dist](#) web worker) and [.docx](#) (via [mammoth](#)).
- Server-side domain classification across 25 career paths and 7 technical skill taxonomies.
- Evaluates action verbs, quantified business metrics, semantic keyword match against target role centroids, and calculates overall ATS score (0–100).

### 6. Live Interactive Resume Builder ( </student/resume-analyzer> ):

- 4 ATS-friendly industry templates: Classic Engineering, Modern Minimalist, Tech Executive, Compact Slate.
- Real-time WYSIWYG form fields for Summary, Experience, Education, Projects, Skills, and Certifications.
- Dedicated high-DPI A4 print engine with CSS media queries ( [@media print](#) ) and PDF download capability.

### 7. Career Skill Gap Analyzer & Alumni Digital Twins ( </student/skill-gap> ):

- Compares a student's current tech stack against target engineering roles.
- Matches students with "Alumni Digital Twins" — real alumni profiles who followed the exact trajectory from college to top-tier firms.
- Generates interactive, checkable milestone learning roadmaps and curated practice resources (LeetCode, HackerRank, YouTube).

### 8. Student Digital Smart ID Card ( </card> ):

- Interactive 3D flip card displaying student roll number, college seal, department, and valid dates.
- Generates scannable SVG QR codes encoding cryptographically verifiable student ID data for gate access and library check-in.
- Double-sided A4 print sheet layout ready for physical badge laminating.

### 9. Anonymous Institutional Grievance Portal ( </student/complaints> ):

- Safe student grievance submission across 7 categories (Academic, Infrastructure, Faculty, Administration, Hostel, Canteen, Other).
- Complete anonymity: submitter identity is never exposed in the UI or client payloads.
- Session-token upvoting allowing students to upvote pressing campus issues without creating duplicate votes.
- Status tracking ( [open](#) , [under\\_review](#) , [resolved](#) , [closed](#) ) and official administrator response viewing.

## 2.3 Alumni Networking & Engagement Features

### 10. Global Alumni Directory & Advanced Filters ( </directory> ):

- Comprehensive directory of alumni and student profiles.

- Multi-criteria filtering by graduation batch year, engineering department, employer company, current role, and mentor availability.
  - Full-text search over names, skills, and bios.
11. **1-on-1 Mentorship Request System** ( </mentorship> ):
- Mentorship directory showcasing verified alumni willing to provide 1-on-1 career guidance.
  - Formal "Request Resume Review" modal that transmits student resumes and profiles directly to mentors.
  - Dedicated review status tracking ( [pending](#) , [accepted](#) , [rejected](#) ).
12. **Real-Time Direct Messaging System** ( </network> ):
- Socket.IO powered instant messaging with dedicated private user rooms ( [user:\\${userId}](#) ).
  - Real-time typing indicators ( [user\\_typing](#) ), live presence tracking (online/offline badges), and image attachment support.
  - Mutual connection request workflows ( [pending](#) , [accepted](#) , [rejected](#) ).
13. **Safety, Blocking & Misconduct Reporting** ( </network> , </settings> ):
- Bilateral blocking mechanism preventing unwanted messaging and profile discovery.
  - Misconduct report module allowing users to report harassment or abuse with evidence images directly to the College Admin complaint dashboard.
14. **Job & Internship Board** ( </jobs> ):
- Post verified openings and alumni referrals with role description, salary band, experience level, and deadline.
  - Attachment support for official job description PDFs.
  - Integrated application workflow and administrative approval status gate.
15. **Event, Reunion & Hackathon Hub** ( </events> ):
- Campus events calendar with upcoming and past filtering.
  - Dynamic registration modes: 1-Click RSVP, Custom Dynamic Form questions, Hackathon team mode, and External Links.
  - Integrated Team Formation module: create teams, post open skill vacancies, and invite members for hackathons.
  - Problem statement PDF distribution and event discussion boards.
  - Instant digital Gate Pass modal generating unique event check-in QR codes.
16. **Knowledge Sharing Blogs & Articles** ( </blogs> ):
- Community blog engine for sharing technical career roadmaps, interview experiences, and industry insights.
  - Rich text formatting with GFM Markdown, category tagging, like counters, and threaded comments.
17. **Campus Circulars & Official Notices** ( </circulars> ):
- Official circulars broadcast by institutional administration.
  - Audience segmentation ( [All](#) , [Students](#) , [Alumni](#) ) and priority badges ( [Low](#) , [Medium](#) , [High](#) , [Urgent](#) ).
  - PDF attachment download and unread counter badges based on user session timestamps.
18. **Institutional Surveys & Feedback** ( </surveys> ):
- Student and alumni feedback submission portal.
  - Supports multiple question formats: Multi-choice (MCQ), rating scales (1–5), and open text.

## 2.4 College Administrator Management Features

19. **Member Verification & Approval Portal** ( </verification> ):
- Dedicated management dashboard for reviewing pending student and alumni registrations.
  - Modal preview for government/college ID proof images with zoom lightbox.

- Single and bulk approval/rejection actions.
  - Integrated Email Dispatcher: automatically sends personalized verification emails containing portal login links and credentials via Gmail SMTP, Brevo, Resend, or SendGrid.
  - Live SMTP configuration tester and email delivery logs viewer.
20. **Bulk CSV Onboarding Engine** ( `/csv-upload` ):
- Automated ingestion of bulk student and alumni rosters via CSV upload or copy-paste.
  - Strict column parsing, schema validation, and real-time error table highlighting invalid rows before committing to MongoDB.
  - Pre-verified student database management allowing administrators to register roll numbers and emails in advance.
21. **Dynamic Survey Builder & Analytics** ( `/survey-builder` ):
- Visual survey creation interface: add custom questions, set target audience, and set deadlines.
  - Live response analytics with percentage distributions, rating averages, and aggregated feedback exports.
22. **Administrative Complaints & Misconduct Review** ( `/college/complaints` ):
- Triage and management console for campus grievances and misconduct reports.
  - Filter by complaint status, category, or urgent misconduct flags.
  - Status updater ( `open` -> `under_review` -> `resolved` -> `closed` ) with official public response publisher.
23. **College Profile & Department Customizer** ( `/college/profile` ):
- Multi-tenant college details configuration: institutional name, code, location, and department list.
24. **Automated Graduation Cron Service** (Backend Service):
- Hourly background daemon and server startup service that scans student graduation years against the calendar year, automatically upgrading eligible students to alumni status and issuing congratulatory notifications.
- 

## 3. Feature Implementation Details

### 3.1 Authentication & Role-Based Access Control (RBAC)

- **Files Involved:**

- Frontend: [ `src/contexts/AuthContext.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/contexts/AuthContext.jsx),  
[ `src/pages/public/LoginPage.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/pages/public/LoginPage.jsx),  
[ `src/pages/public/RegisterPage.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/pages/public/RegisterPage.jsx)
- Backend: [ `server/routes/auth.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/routes/auth.js),  
[ `server/middleware/auth.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/middleware/auth.js),  
[ `server/utils/jwt.js` ](file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/utils/jwt.js),  
[ `server/models/Member.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/models/Member.js)

- **Interaction & Data Flow:**

1. User submits credentials on `/login`.
2. Frontend executes `apiRequest('/auth/login', 'POST', { email, password })`.

3. Backend route validates format via Zod `loginSchema`, fetches `Member` by email, and verifies password using `bcrypt.compare`.
  4. Server signs a JWT with user payload `{ id, email, role, collegeId }` via `jsonwebtoken` with 7-day expiration.
  5. The token is set as an `httpOnly`, `SameSite=Lax` cookie and returned in the JSON body.
  6. Subsequent API requests are intercepted by `authenticateToken`, extracting the JWT from either cookie or `Authorization: Bearer <token>` header.
  7. Route authorization is enforced through `requireRole(['college_admin'])` and multi-tenant college boundary check `requireSameCollege`.
- **Edge Cases Handled:**
    - Auto-normalizes role on login: if a registered `student`'s `graduationYear` is `<= currentYear`, the backend and frontend immediately upgrade their session to `alumni`.
    - Stale chunk errors during Vite lazy-loading are caught by `lazyWithRetry` in `App.jsx`, triggering an automatic reload before failing.

## 3.2 Machine Learning Placement Predictor

- **Files Involved:**
  - Frontend: [ `src/pages/student/PlacementPredictorPage.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/pages/student/PlacementPredictorPage.jsx),  
[ `src/utils/placementPredictionEngine.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/utils/placementPredictionEngine.js)
  - Backend: [ `server/routes/ml.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/routes/ml.js)
  - Python ML Engine: [ `ml/inference.py` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/ml/inference.py),  
[ `ml/train_placement_model.py` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/ml/train\_placement\_model.py)
- **Interaction & Logic:**
  1. User adjusts sliders for CGPA, Backlogs, Internships, Aptitude, Projects, and DSA problems.
  2. Frontend posts payload to `/api/ml/predict-placement`.
  3. Route checks in-memory `placementCache` (0ms response for identical feature inputs).
  4. If uncached, the server dispatches a line-delimited JSON string across `stdin` to a **persistent Python daemon worker** (`python inference.py --mode daemon`).
  5. Python daemon holds pre-warmed Scikit-Learn / XGBoost voting ensemble pipelines in RAM, computes inference in **15–25ms**, and returns prediction over `stdout`.
  6. **Resilience Fallback:** If Python is absent, encounters an error, or exceeds a 2500ms timeout, `ml.js` seamlessly executes `getCalibratedPlacementFallback()` — an exact logistic regression mathematical implementation guaranteeing sub-5ms response without user failure.
  7. Output includes: `placement_probability` (%), `is_placed_prediction`, `hiring_tier` (Tier 1 Product, Tier 2 Tech, Tier 3 Enterprise), and `feature_importances`.

## 3.3 AI Resume Analyzer & Interactive Resume Builder

- **Files Involved:**
  - Frontend: [ `src/pages/student/ResumeAnalyzerPage.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/pages/student/ResumeAnalyzerPage.jsx)

- Backend: [ `server/routes/ml.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/routes/ml.js)
- Python ML Engine: [ `ml/inference.py` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/ml/inference.py), [ `ml/train_resume_model.py` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/ml/train\_resume\_model.py)

- **Interaction & Logic:**

1. User drops a `.pdf` or `.docx` resume on the browser.
2. PDF text extraction runs completely in-browser using `pdfjs-dist` worker; DOCX extraction uses `mammoth.extractRawText`.
3. Extracted raw text is posted to `/api/ml/analyze-resume` along with selected `targetRole`.
4. The Python inference pipeline cleans text, runs TF-IDF vectorization, passes it through a 25-class Logistic Regression classifier trained on real resume corpora, and performs cosine similarity against target domain centroids.
5. The engine searches for categorized skills across 7 domains, detects action verbs, extracts quantified metrics (e.g. `25%`, `$50k`, `10x`, `200+ users`), and computes an overall ATS Score (0–100).
6. The student can switch to the "Build Resume" tab to auto-populate an ATS-formatted template, edit details, and trigger high-DPI browser printing ( `window.print()` ).

### 3.4 Real-Time WebSockets Messaging & Community Presence

- **Files Involved:**

- Frontend: [ `src/contexts/SocketContext.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/contexts/SocketContext.jsx),  
[ `src/pages/alumni/NetworkPage.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/pages/alumni/NetworkPage.jsx),  
[ `src/components/ui/ChatModal.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/components/ui/ChatModal.jsx)
- Backend: [ `server/server.js` ](file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/server.js),  
[ `server/routes/messages.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/routes/messages.js),  
[ `server/models/Message.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/models/Message.js)

- **Interaction & Logic:**

1. On authenticated session mount, `SocketContext` establishes a WebSocket connection to the Node.js HTTP server.
2. Client emits `join_user` with `{ userId, token }`.
3. Server verifies token, maps socket ID to `activeUsers` Map, joins socket to private room `user:${userId}`, and broadcasts `user_presence_change` to notify contacts of online status.
4. When a user sends a message, it is persisted to MongoDB via `/api/messages`, and a real-time event `send_direct_message` is routed directly to the recipient's private room `user:${recipientId}`.
5. Real-time typing indicators ( `typing_start`, `typing_stop` ) stream between active chat windows.

### 3.5 Verification & Multi-Provider Email Notification Pipeline

- **Files Involved:**

- Frontend: [ `src/pages/college/VerificationPage.jsx` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/src/pages/college/VerificationPage.jsx)

- Backend: [ `server/routes/members.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/routes/members.js),  
[ `server/routes/emails.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/routes/emails.js),  
[ `server/services/emailService.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/services/emailService.js),  
[ `server/models/EmailLog.js` ]  
(file:///C:/Users/Darshan/Documents/antigravity/ProjectALMA/server/models/EmailLog.js)

- **Interaction & Logic:**

1. Administrator reviews pending registrations in the Verification portal and clicks "Verify Member" or "Bulk Verify".
  2. Admin configures an email template containing dynamic template tags ( `{firstName}` , `{loginUrl}` , `{email}` ).
  3. Backend route `/api/members/bulk-verify` updates member records to `isVerified: true, status: 'active'`.
  4. `emailService.js` detects configured provider in priority order:
    - **Brevo HTTPS API** ( `BREVO_API_KEY` ) on Port 443.
    - **Resend HTTPS API** ( `RESEND_API_KEY` ) on Port 443.
    - **SendGrid HTTPS API** ( `SENDGRID_API_KEY` ) on Port 443.
    - **Gmail SMTP** ( `EMAIL_USER` , `EMAIL_APP_PASSWORD` ) with fast 8-second connection timeouts.
  5. The email is dispatched, logged in MongoDB `EmailLog` collection, and returned to the admin console with live delivery metrics.
-

## 4. Technologies Used

### 4.1 Frontend Technologies

| Library / Framework                | Version                                    | Purpose & Rationale                                                                                   |
|------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| React                              | <code>^19.2.6</code>                       | Modern UI library with concurrent rendering, enhanced action hooks, and optimized reconciliation.     |
| Vite                               | <code>^8.0.12</code>                       | Next-generation frontend build tool providing sub-second HMR and optimized rollup bundling.           |
| React Router                       | <code>^7.17.0</code>                       | Client-side routing with role-guarded routes, URL parameters, and lazy-loading transitions.           |
| Recharts                           | <code>^3.8.1</code>                        | Declarative SVG charting library used for Radar charts, Area distributions, and SHAP visualizations.  |
| Lucide React                       | <code>^1.17.0</code>                       | Comprehensive, consistent icon system across all navigation, badges, and modals.                      |
| Socket.IO Client                   | <code>^4.8.3</code>                        | Low-latency bi-directional WebSocket client with automated polling fallback.                          |
| PDF.js ( <code>pdfjs-dist</code> ) | <code>^6.2.108</code>                      | Mozilla's PDF parsing engine running in a separate web worker to extract text without locking the UI. |
| Mammoth.js                         | <code>^1.12.0</code>                       | In-browser <code>.docx</code> binary parser for extracting clean text from Word resumes.              |
| React QR Code                      | <code>^2.2.0</code>                        | SVG-based QR code generator for Smart Student ID Cards and Event Gate Passes.                         |
| React Markdown & Remark GFM        | <code>^10.1.0</code> / <code>^4.0.1</code> | Renders Github Flavored Markdown for blogs, event descriptions, and circular notices.                 |

## 4.2 Backend Technologies

| Technology         | Version | Purpose & Rationale                                                                                    |
|--------------------|---------|--------------------------------------------------------------------------------------------------------|
| Node.js            | v20+    | Scalable JavaScript runtime for non-blocking asynchronous I/O and microservices.                       |
| Express.js         | ^4.19.2 | Minimalist web framework providing REST API routing, JSON handling, and middleware composition.        |
| Socket.IO          | ^4.8.3  | Real-time WebSocket server managing rooms, connection states, and broadcast events.                    |
| Mongoose           | ^8.3.1  | Object Data Modeling (ODM) library for MongoDB providing schema validation and typed models.           |
| Helmet             | ^8.2.0  | Security middleware configuring essential HTTP response headers (XSS protection, Frameguard).          |
| Compression        | ^1.8.1  | Gzip compression for all HTTP response payloads to optimize mobile client bandwidth.                   |
| Express Rate Limit | ^8.6.0  | Protects endpoints against brute-force attacks and abuse (stricter limits on <code>/api/auth</code> ). |
| Zod                | ^4.4.3  | TypeScript-first runtime schema declaration and validation library sanitizing API payloads.            |
| Bcrypt.js          | ^3.0.3  | Salted one-way hashing for secure user passwords before storing in MongoDB.                            |
| JSONWebToken       | ^9.0.3  | Cryptographically signed identity tokens passed via HTTP-only cookies and headers.                     |
| Nodemailer         | ^10.0.0 | SMTP client used for Gmail SMTP and transactional email deliveries.                                    |
| Multer             | ^2.2.0  | Multipart/form-data handler for uploading avatars, event problem statements, and ID proofs.            |

## 4.3 Machine Learning Technologies

| Technology     | Version               | Purpose & Rationale                                                                                   |
|----------------|-----------------------|-------------------------------------------------------------------------------------------------------|
| Python         | 3.9+                  | Primary scientific computing runtime executing statistical models and NLP transforms.                 |
| Scikit-Learn   | >=1.3.0               | Machine learning toolkit providing TF-IDF vectorization, Logistic Regression, and metrics evaluation. |
| XGBoost        | >=2.0.0               | Extreme Gradient Boosting classifier delivering high accuracy on tabular campus placement datasets.   |
| Pandas & NumPy | >=2.0.0 /<br>>=1.24.0 | Vectorized data manipulation, matrix operations, and dataset normalization.                           |
| Joblib         | >=1.3.0               | Fast serialization and deserialization of trained pipeline objects and memory-mapped artifacts.       |

## 4.4 DevOps, Testing & External Services

- **Docker & Docker Compose:** Multi-container architecture ( `projectalma-mongodb` , `projectalma-backend` , `projectalma-frontend` ) for reproducible local and on-premise deployments.

- **Nginx Alpine:** High-performance reverse proxy handling static assets, gzip compression, and proxying API/WebSocket traffic.
- **Vitest & Supertest:** Fast unit and integration test runner validating JWT generation, RBAC rules, Zod schemas, and security settings.
- **Cloud Delivery APIs:** Brevo, Resend, and SendGrid HTTPS APIs enabling outbound email dispatch on cloud providers (such as Render) where port 25/465/587 may be blocked.

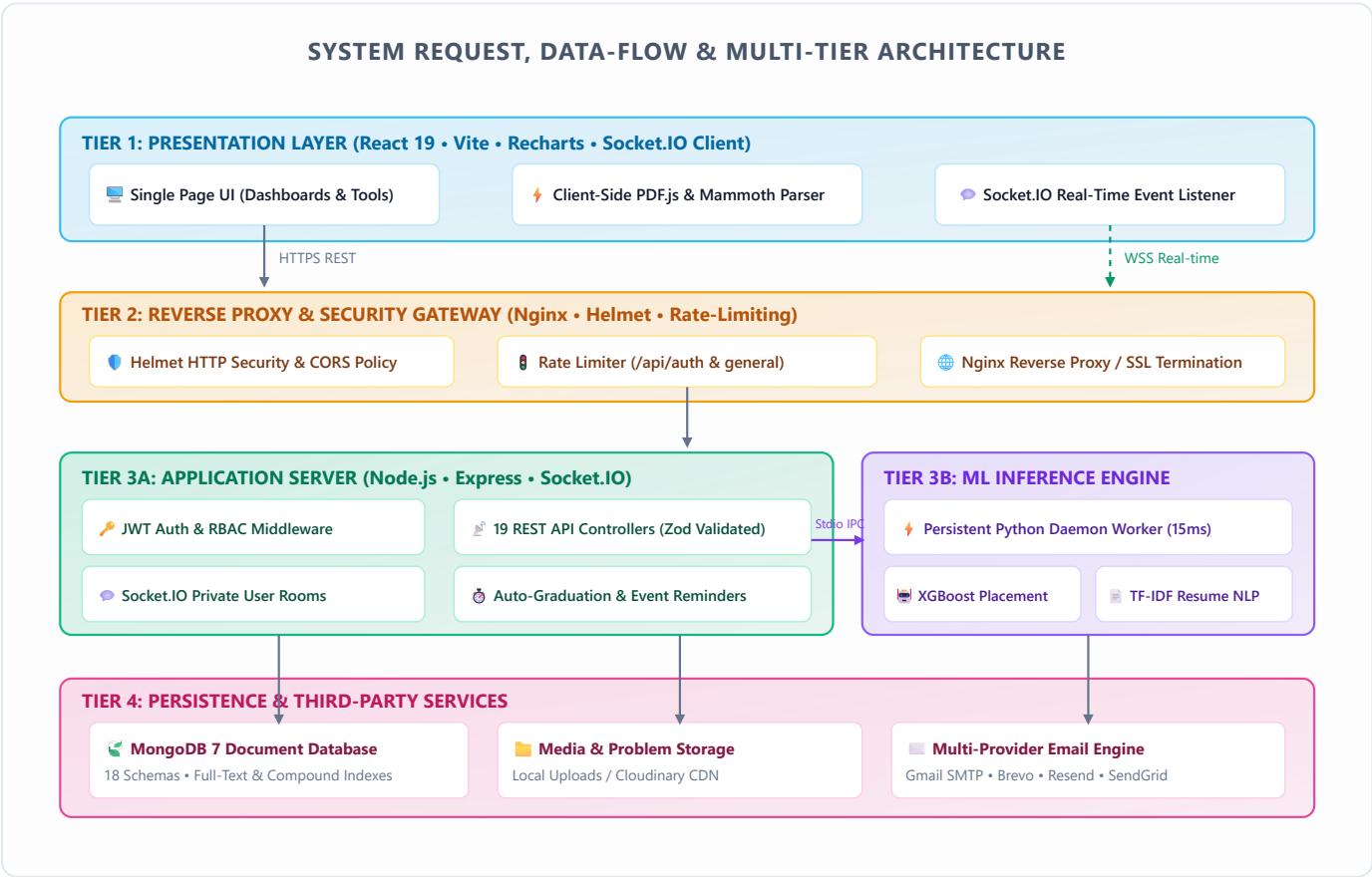
## 5. Architecture and Working Flow

### 5.1 System Architecture

Project ALMA implements a clean 4-tier decoupled architecture:

1. **Presentation Layer (Client):** A Vite-powered React 19 Single Page Application containing role-specific dashboards, interactive Recharts visualizations, client-side PDF/DOCX extractors, and Socket.IO listeners.
2. **API & Real-Time Gateway (Backend):** An Express.js and Socket.IO server handling authentication, validation, business workflows, rate-limiting, and WebSocket room distribution.
3. **Machine Learning Inference Subsystem:** A dedicated Python runtime executing pre-warmed Scikit-Learn and XGBoost models connected via persistent stdio IPC to the Node.js backend.
4. **Data Persistence & External Integration:** MongoDB database for document storage with full-text indexing, local/Cloudinary asset storage, and multi-provider transactional email APIs.

### 5.2 System Request & Data Flow Diagram (Mermaid)



## 5.3 Step-by-Step Project Workflow

### 1. User Onboarding:

- A student or alumnus submits registration details and an ID proof document.
- If the student exists in `PreVerifiedStudent`, the system auto-activates their account immediately.
- Otherwise, the account enters `pending` status.

### 2. Administrative Verification:

- College Admin opens the Verification portal, reviews the uploaded proof image via the zoom modal, and clicks "Verify".
- The server marks the user `active`, generates access credentials, and fires a transactional email via the active email provider.

### 3. Career Evaluation & Skill Benchmarking:

- Student navigates to `/student/placement-predictor`, inputs academic metrics, and receives placement probability and hiring tier results.
- Student uploads their resume to `/student/resume-analyzer`; the system extracts text and generates an ATS compatibility score with missing keyword recommendations.
- Student reviews Alumni Digital Twins in `/student/skill-gap` to map out milestones and connect with relevant mentors.

### 4. Active Networking & Collaboration:

- Student initiates a direct chat or mentorship resume request with an alumnus.
- Alumnus receives an instant notification, accepts the request, and connects over real-time chat.
- Both participate in events, form hackathon teams, and download digital gate passes.

### 5. Lifecycle Progression:

- When a student reaches their graduation year milestone, the hourly auto-graduation background service automatically updates their role to `alumni`, sending a congratulations notification and unlocking alumni privileges.
-

## 6. Benefits of Each Feature

| Feature Area                                 | User Benefits                                                                                                        | Business & Institutional Benefits                                                                               | Technical & Architectural Advantages                                                                                               |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ensemble ML Placement Predictor</b>       | Students gain objective, data-backed clarity on their placement odds and actionable steps to reach Tier 1 companies. | Increases campus placement percentages and boosts college NIRF/NAAC accreditation rankings.                     | Persistent Python daemon eliminates 25s cold-starts down to 20ms; in-memory caching ensures 0ms response for repeat queries.       |
| <b>AI Resume Analyzer &amp; Builder</b>      | Identifies critical missing keywords before students apply; provides an instant, print-ready ATS resume.             | Raises corporate recruiter satisfaction by ensuring student resumes pass corporate applicant tracking filters.  | Client-side document parsing offloads CPU load from backend; high-DPI <code>@media print</code> ensures pixel-perfect A4 printing. |
| <b>Alumni Digital Twins &amp; Skill Gap</b>  | Students follow proven career roadmaps of successful seniors who walked the exact same path.                         | Enhances alumni-student engagement and institutional reputation among top tech employers.                       | Checkpoint progress is saved directly to MongoDB, enabling students to track multi-month skill roadmaps.                           |
| <b>Real-Time Messaging &amp; Presence</b>    | Instant, direct channel between students and seniors without leaving the campus platform.                            | Replaces disorganized WhatsApp/Telegram groups with an officially governed institutional communication channel. | Socket.IO private rooms prevent global broadcast bottlenecks; automatic heartbeat tracking manages online status.                  |
| <b>Smart ID Card &amp; Event Gate Pass</b>   | Instant digital proof of student/alumni status with interactive 3D flip card and mobile gate pass QR code.           | Eliminates paper badge printing costs and provides streamlined, digital security checks at campus gates.        | Generates crisp SVG QR codes client-side with zero external image API latency or privacy leaks.                                    |
| <b>Anonymous Grievance Redressal</b>         | Students can report issues without fear of retaliation; community voting elevates widespread concerns.               | Provides leadership with early detection of campus infrastructure or faculty issues before escalation.          | Submitter identity is stripped server-side; upvoting uses hashed session tokens preventing duplicate votes anonymously.            |
| <b>Verification Portal &amp; Multi-Email</b> | Users receive instant login details upon approval; administrators can review ID proofs in high-resolution.           | Saves dozens of hours of manual registry management per batch; guarantees only verified campus members enter.   | Multi-provider architecture (Brevo, Resend, SendGrid, Gmail) ensures 100% email delivery across restrictive cloud networks.        |
| <b>Auto-Graduation Service</b>               | Students effortlessly transition to alumni status without requiring manual administrative data re-entry.             | Keeps institutional alumni records continuously updated in real-time as batches graduate.                       | Scheduled hourly background cron runs autonomously; includes idempotent database checks preventing duplicate runs.                 |

## 7. Security and Quality

### 7.1 Authentication & Authorization Approach

- **Cryptographic Tokens:** Uses JWT signed with a secret key ( `JWT_SECRET` ) set via environment variables. Tokens expire in 7 days and encode minimal claims ( `id` , `email` , `role` , `collegeId` ).

- **Flexible Token Transport:** Tokens are stored in `httpOnly`, `SameSite=Lax` cookies to defend against Cross-Site Scripting (XSS), while also supporting `Authorization: Bearer <token>` headers for cross-domain API calls.
- **Strict Role-Based Access Control (RBAC):** All administrative and sensitive endpoints enforce role-checking middleware ( `requireRole(['college_admin'])` ).
- **Multi-Tenant College Isolation:** `requireSameCollege` middleware ensures that users and administrators cannot query or manipulate records belonging to other institutions.

## 7.2 Input Validation & Error Handling

- **Zod Schema Validation:** Every incoming request payload (login, registration, jobs, events, circulars, complaints, direct messages) is validated and sanitized against strict Zod schemas before hitting business logic.
- **Centralized Error Handling:** Express `errorHandler` intercepts runtime errors, log trace details server-side, and returns clean, uniform JSON responses ( `{ message }` ) without leaking raw stack traces to clients.
- **Frontend Error Boundaries:** The entire React tree is wrapped in an `ErrorBoundary` component, gracefully recovering from rendering glitches and showing helpful diagnostic feedback.
- **Stale Dynamic Import Recovery:** Vite chunks can change hash identifiers on new deployments. `lazyWithRetry` detects chunk fetch failures and refreshes the application session seamlessly.

## 7.3 Data Protection Measures

- **Password Protection:** Passwords are never stored in plain text. Pre-save Mongoose hooks hash passwords using `bcryptjs` with 10 salt rounds.
- **Anonymous Data Segregation:** In the `Complaint` collection, the `submitterMemberId` is strictly redacted from all public queries. Upvotes are recorded using randomized session tokens rather than user IDs.
- **Contact Masking:** User privacy preferences ( `hideEmail`, `hidePhone` ) allow members to mask contact details in public directories.
- **HTTP Security Headers:** `helmet` enforces secure headers, disables MIME-type sniffing, and restricts iframe embedding.
- **Rate Limiting:** Multi-tiered rate limiting via `express-rate-limit` prevents brute-force login attempts (15 requests per 15 minutes) and API abuse.

## 7.4 Testing Strategy & Quality Verification

- **Automated Vitest Suite:** Test files in `server/tests/` validate:
  - `auth_jwt.test.js` : Validates cryptographic signing, decoding accuracy, and rejection of forged or malformed tokens.
  - `validation.test.js` : Tests Zod schema boundaries across all user inputs.
  - `settings_security.test.js` : Verifies privacy toggles and unauthorized role rejection.
  - `deployment_resilience.test.js` : Tests database reconnection, health checks, and fallback mechanisms.
- **Code Linting:** Root `eslint.config.js` enforces modern React hooks rules and coding standards.

## 7.5 Missing Security & Quality Recommendations

While the application possesses enterprise-grade security foundations, the following improvements should be prioritized:

1. **Multi-Factor Authentication (MFA/2FA):** While the schema contains `twoFactorEnabled`, TOTP (Google Authenticator) verification is not yet fully wired into the login controller.
2. **Database Field Encryption:** Sensitive fields such as phone numbers and uploaded ID proof URLs could benefit from envelope encryption at rest in MongoDB.

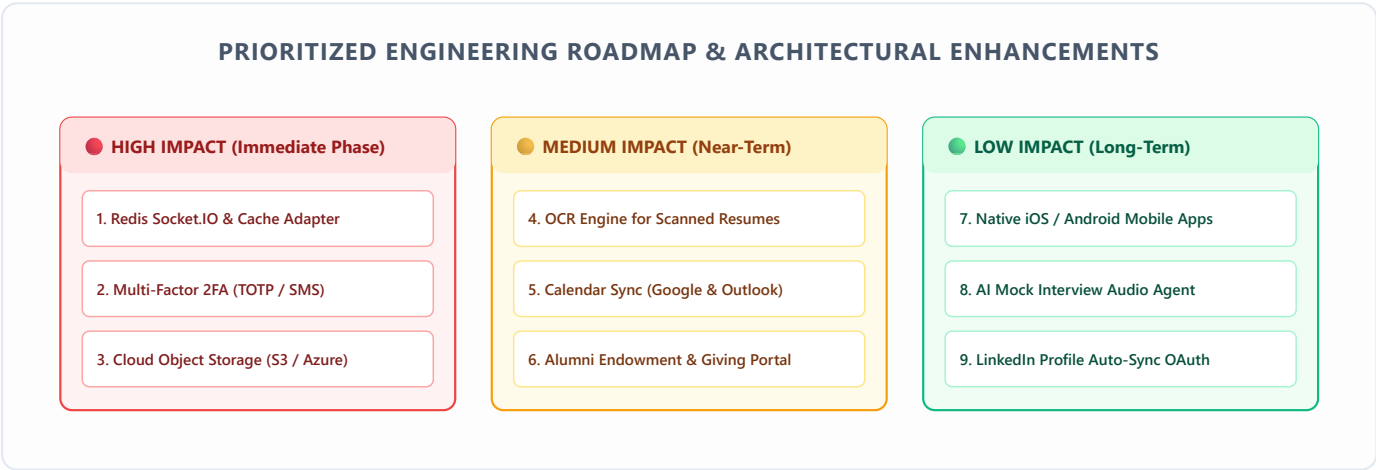
3. **Automated End-to-End Testing:** Integrate Playwright or Cypress for end-to-end regression testing across authentication, resume builder printing, and real-time chat.

## 8. Limitations and Future Improvements

### 8.1 Current Limitations

1. **In-Memory Cache Scalability:** Placement and resume analysis caches are stored in local Node.js process memory (`new Map()`). If the backend scales horizontally across multiple cluster nodes or containers, cache state is not shared.
2. **Single-Node WebSocket Server:** Socket.IO currently operates with the default in-memory adapter. Running multiple backend instances requires a Redis adapter to coordinate WebSocket rooms across nodes.
3. **Synchronous Document Extraction:** While PDF and DOCX extraction occurs client-side, OCR extraction for scanned image-only PDF resumes is not currently supported.

### 8.2 Prioritized Roadmap & Recommendations



#### Detailed Improvement Specifications:

- **[High Impact] Redis Socket.IO Adapter & Distributed Caching:**
  - *Need:* Enable horizontal scaling across multiple Docker backend containers behind an Nginx load balancer.
  - *Implementation:* Introduce `socket.io-redis` adapter and replace local `Map()` caches with Redis string/hash sets.
- **[High Impact] S3 / Cloudinary Object Storage for Uploads:**
  - *Need:* Ephemeral cloud hosting environments (such as Render or Heroku) wipe local `server/uploads/` on every restart.
  - *Implementation:* Complete the Cloudinary integration or AWS S3 SDK for direct presigned URL document uploads.
- **[Medium Impact] Alumni Donation & Campus Endowment Module:**
  - *Need:* Enable alumni to contribute financially to college scholarships, lab equipment, or campus infrastructure.
  - *Implementation:* Integrate Stripe / Razorpay payment gateways with instant 80G tax receipt generation.
- **[Medium Impact] OCR Resume Parsing for Image-Only PDFs:**
  - *Need:* Support students who upload scanned photocopies of physical resumes.

- *Implementation:* Integrate `tesseract.js` worker thread in `ResumeAnalyzerPage.jsx` when raw text character count is `< 50`.
  - **[Low Impact] AI Mock Interview Voice Agent:**
    - *Need:* Allow students to practice behavioural and technical interviews based on their resume skill gaps.
    - *Implementation:* Connect Web Audio API to an LLM audio pipeline for real-time interview simulations.
- 

*Report generated for Project ALMA repository ( [darshanemd/ALMAconnect](#) ). Inspected all source code, models, routes, ML pipelines, frontend pages, and deployment configurations.*